

JavaScript Event Propagation — 40 Interview Questions and Answers

1. What is event propagation in JavaScript?

Event propagation defines how an event travels through the DOM tree in three phases:

1. **Capturing phase:** Event travels from the top (window/document) down to the target.
 2. **Target phase:** Event reaches the element that triggered it.
 3. **Bubbling phase:** Event bubbles up from the target to its ancestors.
-

2. What are capturing and bubbling phases?

- **Capturing:** Event moves from root → target (top-down).
- **Bubbling:** Event moves from target → root (bottom-up).

js

 Copy code

```
parent.addEventListener('click',()=>console.log('capturing'),true);
child.addEventListener('click',()=>console.log('bubbling'));
```

Click child →

nginx

 Copy code

```
capturing
bubbling
```

3. What is the target phase?

The phase where the event has reached the actual element that triggered it. Both capturing and bubbling listeners can run here.

4. What's the default phase for event listeners?

By default, event listeners work in the **bubbling phase** (`useCapture` = false).

5. What's the syntax for adding an event listener?

js

 Copy code

```
element.addEventListener(eventType, listener, useCapture);
```

- `eventType` : event name like `"click"` .
- `listener` : function to execute.
- `useCapture` : `true` for capturing, `false` for bubbling.

6. How to identify which element triggered the event?

Use `event.target` .

js

 Copy code

```
document.body.addEventListener('click', e => {  
  console.log('Clicked element:', e.target.tagName);  
});
```

7. What is `event.currentTarget` ?

The element whose listener is currently running.

js

 Copy code

```
list.addEventListener('click', e => {  
  console.log('target:', e.target.id, 'currentTarget:', e.currentTarget.id);  
});
```

8. What's the difference between `target` and `currentTarget` ?

Property	Description
<code>target</code>	Element that initiated the event.

Property	Description
<code>currentTarget</code>	Element whose listener is executing.

9. What does `event.eventPhase` represent?

It indicates which phase is currently active:

- 1 = Capturing
 - 2 = Target
 - 3 = Bubbling
-

10. How can you stop propagation?

Use `event.stopPropagation()` to stop the event from traveling further.

js

 Copy code

```
child.addEventListener('click', e => {  
  e.stopPropagation();  
  console.log('Child only');  
});
```

11. How to stop all other listeners on the same element?

Use `event.stopImmediatePropagation()`.

js

 Copy code

```
btn.addEventListener('click', ()=>console.log('A'));  
btn.addEventListener('click', (e)=>{e.stopImmediatePropagation();console.log('B');});  
btn.addEventListener('click', ()=>console.log('C'));
```

Output → `B`.

12. Difference between `stopPropagation()` and `preventDefault()`?

- `stopPropagation()` → stops event flow.
- `preventDefault()` → prevents browser's default behavior (e.g., following a link).

js

 Copy code

```
a.addEventListener('click', e => {  
  e.preventDefault();  
  console.log('Link not opened');  
});
```

13. Can we use both capturing and bubbling on same element?

Yes.

js

 Copy code

```
div.addEventListener('click', ()=>console.log('capture'),true);  
div.addEventListener('click', ()=>console.log('bubble'));
```

Output:

nginx

 Copy code

```
capture  
bubble
```

14. Give an example of event bubbling.

html

 Copy code

```
<div id="parent">  
  <button id="child">Click</button>  
</div>  
  
<script>  
  parent.addEventListener('click', ()=>console.log('parent clicked'));  
  child.addEventListener('click', ()=>console.log('child clicked'));  
</script>
```

Click child →

nginx

 Copy code

```
child clicked  
parent clicked
```

15. Example of event capturing.

js

 Copy code

```
parent.addEventListener('click',()=>console.log('parent capture'),true);  
child.addEventListener('click',()=>console.log('child bubble'));
```

Output:

nginx

 Copy code

```
parent capture  
child bubble
```

16. What are non-bubbling events?

`focus`, `blur`, `mouseenter`, `mouseleave`, `load`, `unload`, `scroll`, `resize`.

17. How to make focus events bubble?

Use `focusin` and `focusout`.

js

 Copy code

```
document.body.addEventListener('focusin', e => console.log('Focused:', e.target));
```

18. What is event delegation?

It's when a parent element handles events from its children through bubbling.

js

 Copy code

```
ul.addEventListener('click', e => {  
  if(e.target.tagName === 'LI') console.log('Clicked:', e.target.textContent);  
});
```

19. Why is event delegation efficient?

It reduces memory usage by using one handler for many elements and works even for dynamically added elements.

20. How does event order behave with multiple listeners on the same element?

They execute in the order they were registered unless `stopImmediatePropagation()` is called.

21. How to remove a listener properly?

You must use the same function reference and capture flag.

js

 Copy code

```
function handler(){console.log('click');}  
btn.addEventListener('click', handler, true);  
btn.removeEventListener('click', handler, true);
```

22. What does `dispatchEvent()` do?

It manually fires an event.

js

 Copy code

```
const event = new Event('hello');  
el.dispatchEvent(event);
```

23. What is the return value of `dispatchEvent()`?

`true` if no listener called `preventDefault()`, else `false`.

24. What are cancelable events?

Events that allow their default action to be canceled (e.g., `click`, `submit`, `keydown`).

25. What are passive listeners?

Listeners that promise not to call `preventDefault()`. Improves performance for scroll/touch.

js

 Copy code

```
window.addEventListener('scroll', handle, { passive: true });
```

26. What is the `once` option in `addEventListener()`?

It removes the listener after the first call.

js

 Copy code

```
btn.addEventListener('click', ()=>console.log('first click only'),{once:true});
```

27. What is `AbortController` used for in events?

To remove listeners programmatically.

js

 Copy code

```
const ctrl = new AbortController();
document.addEventListener('click', ()=>console.log('click'),{signal:ctrl.signal});
ctrl.abort(); // removes listener
```

28. Difference between pointer capture and event capturing?

- **Event capturing:** DOM phase from top to target.
- **Pointer capture:** Locks subsequent pointer events to one element.

js

 Copy code

```
el.addEventListener('pointerdown', e => el.setPointerCapture(e.pointerId));
```

29. How do events propagate inside Shadow DOM?

They stop at the shadow boundary unless `composed:true` is set.

js

 Copy code

```
host.dispatchEvent(new CustomEvent('notify',{bubbles:true, composed:true}));
```

30. What does `event.composedPath()` return?

An array of nodes the event passed through (including shadow roots).

31. Example of `event.target` usage.

js

 Copy code

```
document.addEventListener('click', e => {  
  if(e.target.matches('button')) {  
    console.log('Button clicked:', e.target.textContent);  
  }  
});
```

32. When are `target` and `currentTarget` equal?

When the handler is attached directly to the element that triggered the event.

33. Example showing bubbling order.

html

 Copy code

```
<div id="outer">
  <div id="inner">
    <button id="btn">Click</button>
  </div>
</div>

<script>
  outer.addEventListener('click',()=>console.log('outer'));
  inner.addEventListener('click',()=>console.log('inner'));
  btn.addEventListener('click',()=>console.log('btn'));
</script>
```

Click button →

sql

 Copy code

```
btn
inner
outer
```

34. Example showing capturing order.

js

 Copy code

```
outer.addEventListener('click',()=>console.log('outer capture'),true);
inner.addEventListener('click',()=>console.log('inner capture'),true);
btn.addEventListener('click',()=>console.log('btn'));
```

Output:

sql

 Copy code

```
outer capture
inner capture
btn
```

35. Real-world use of stopPropagation() — Modal example.

js

 Copy code

```
backdrop.addEventListener('click',()=>closeModal());  
modal.addEventListener('click',e=>e.stopPropagation());
```

Stops closing when clicking inside modal.

36. How does React handle event propagation?

React uses **Synthetic Events** and one global listener at the root.

`onClickCapture` → capturing, `onClick` → bubbling.

37. Example of React capturing and bubbling.

jsx

 Copy code

```
<div onClickCapture={()=>console.log('capture')} onClick={()=>console.log('bubble')}>  
  <button>Click</button>  
</div>
```

Output:

nginx

 Copy code

```
capture  
bubble
```

38. Example of event delegation using data attributes.

js

 Copy code

```
table.addEventListener('click', e => {  
  const btn = e.target.closest('[data-action]');  
  if(!btn) return;
```

```
console.log('Action:', btn.dataset.action);  
});
```

39. Example to prevent background click-through (overlay issue).

js

 Copy code

```
overlay.addEventListener('click', e => {  
  if(e.target === overlay) closeOverlay();  
});
```

40. Example showing composed and bubbling in a custom event.

js

 Copy code

```
const evt = new CustomEvent('notify', { bubbles: true, composed: true });  
child.dispatchEvent(evt);
```

This allows the event to cross shadow DOM boundaries and bubble up to ancestors.